

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, DECANA DE AMÉRICA)

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Software

GUÍA DE LABORATORIO**Semana 5****Modelo Multidimensional de Datos para DM Analítico e Inteligencia de Negocios — Esquema de un BI****1. INTRODUCCIÓN**

La presente guía de laboratorio acompaña los contenidos teóricos desarrollados en la Semana 5 de la asignatura de Inteligencia de Negocios. El objetivo central es que el estudiante construya, de manera práctica y paso a paso, un Datamart Analítico de Ventas completo, implementando el modelo multidimensional bajo el esquema estrella propuesto por la metodología de Ralph Kimball.

Para ello se utilizará Python como lenguaje de generación y transformación de datos sintéticos, y Power BI Desktop como herramienta de visualización y análisis OLAP. Al término de la sesión guiada, el estudiante resolverá una actividad propuesta de manera autónoma en un tiempo máximo de 40 minutos.

2. LOGROS DE APRENDIZAJE

- Generar datos sintéticos realistas utilizando la librería Faker y pandas de Python.
- Construir las tablas de dimensión (DIM_TIEMPO, DIM_PRODUCTO, DIM_CLIENTE, DIM_TIENDA, DIM_VENDEDOR) siguiendo el esquema estrella.
- Diseñar y poblar la tabla de hechos FACT_VENTAS respetando la granularidad declarada.
- Importar el modelo dimensional a Power BI Desktop y establecer las relaciones correctas.
- Crear medidas calculadas en lenguaje DAX.
- Aplicar operaciones OLAP (Drill-Down, Slice, Dice) sobre el modelo construido.

3. RECURSOS Y HERRAMIENTAS**3.1 Software requerido**

Herramienta	Uso	Descarga
Python 3.10+	Lenguaje de programación principal	python.org
pip (gestor de paquetes)	Para instalar librerías de Python	Incluido con Python
Jupyter Notebook / VS Code	Entorno de desarrollo recomendado	jupyter.org / code.visualstudio.com
Power BI Desktop	Herramienta de visualización BI	powerbi.microsoft.com (gratuito)

3.2 Librerías Python requeridas

Instale las siguientes librerías antes de iniciar el laboratorio ejecutando en la terminal:

```
pip install pandas numpy faker openpyxl
```

Librería	Versión mín.	Propósito
pandas	1.5+	Manipulación y exportación de DataFrames
numpy	1.23+	Operaciones numéricas y generación de distribuciones
faker	18+	Generación de datos sintéticos realistas en español
openpyxl	3.0+	Exportación a formato Excel (.xlsx)

4. MARCO TEÓRICO RESUMIDO

4.1 El Modelo Multidimensional

El modelo multidimensional organiza los datos en dos tipos de tablas: la tabla de hechos (Fact Table), que contiene los eventos de negocio cuantificables y sus métricas numéricas; y las tablas de dimensión (Dimension Tables), que proveen el contexto descriptivo del análisis. El esquema resultante más común es el esquema estrella, en el que la tabla de hechos ocupa el centro y las dimensiones se ubican a su alrededor conectadas mediante claves foráneas.

4.2 Caso del Laboratorio: COMERCIAL INCA S.A.C.

La empresa ficticia COMERCIAL INCA S.A.C. opera una cadena de tiendas retail en Perú. Cuenta con un sistema ERP que registra cada transacción de venta. El gerente comercial requiere un Datamart de Ventas que permita analizar el comportamiento de ventas por producto, cliente, tienda y período.

Aplicando la metodología de Kimball, el diseño del Datamart es el siguiente:

Paso Kimball	Resultado
1. Proceso de Negocio	VENTAS AL DETALLE de tiendas retail
2. Granularidad	Cada fila = una línea de detalle de boleta/factura (1 producto × 1 tienda × 1 fecha × 1 cliente × 1 vendedor)
3. Dimensiones	DIM_TIEMPO, DIM_PRODUCTO, DIM_CLIENTE, DIM_TIENDA, DIM_VENDEDOR
4. Medidas	monto_bruto_sol, descuento_sol, monto_netto_sol, costo_sol, margen_bruto_sol, igv_sol, cantidad_vendida

5. DESARROLLO PASO A PASO

PASO 1: Configuración del entorno de trabajo

Abra Jupyter Notebook o VS Code y cree un nuevo archivo llamado laboratorio_semana5.py. Luego importe todas las librerías necesarias con el siguiente bloque de código:

```
# =====
#  LABORATORIO SEMANA 5 – INTELIGENCIA DE NEGOCIOS
#  Datamart Analítico de Ventas - COMERCIAL INCA S.A.C.
#  Prof. Juan Gamarra Moreno – UNMSM 2026-1
#  =====

import pandas as pd
import numpy as np
from faker import Faker
from datetime import date, timedelta
import random
import os

# Configurar semilla para reproducibilidad
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
fake = Faker('es_PE') # Datos en español peruano
Faker.seed(SEED)

# Carpeta de salida para los archivos CSV
os.makedirs('datamart_ventas', exist_ok=True)
print('Entorno configurado correctamente.')
```

Verifique que la salida muestre:

```
Entorno configurado correctamente.
```

PASO 2: Construcción de DIM_TIEMPO

La dimensión tiempo es la más importante de cualquier Datamart. Generaremos todas las fechas del período 2022-01-01 al 2025-12-31, enriqueciéndola con atributos derivados que permitirán análisis por día, mes, trimestre, semestre y año.

```
# --- DIM_TIEMPO ---
def crear_dim_tiempo(fecha_inicio='2022-01-01', fecha_fin='2025-12-31'):
    fechas = pd.date_range(start=fecha_inicio, end=fecha_fin, freq='D')
    meses_es = {1:'Enero', 2:'Febrero', 3:'Marzo', 4:'Abril',
                5:'Mayo', 6:'Junio', 7:'Julio', 8:'Agosto',
                9:'Septiembre', 10:'Octubre', 11:'Noviembre', 12:'Diciembre'}
    dias_es = {0:'Lunes', 1:'Martes', 2:'Miércoles', 3:'Jueves',
              4:'Viernes', 5:'Sábado', 6:'Domingo'}
    df = pd.DataFrame({
        'id_tiempo'      : fechas.strftime('%Y%m%d').astype(int),
        'fecha'          : fechas.date,
        'dia'            : fechas.day,
        'nombre_dia'     : fechas.dayofweek.map(dias_es),
        'semana'         : fechas.isocalendar().week.astype(int),
        'mes'            : fechas.month,
        'nombre_mes'     : fechas.month.map(meses_es),
        'trimestre'      : fechas.quarter,
        'semestre'       : np.where(fechas.month <= 6, 1, 2),
        'anio'           : fechas.year,
        'es_fin_semana'  : (fechas.dayofweek >= 5).astype(int),
        'es_feriado'     : 0, # simplificado
    })
    return df

dim_tiempo = crear_dim_tiempo()
dim_tiempo.to_csv('datamart_ventas/DIM_TIEMPO.csv', index=False)
print(f'DIM_TIEMPO: {len(dim_tiempo)} filas generadas.')
print(dim_tiempo.head(3).to_string())
```

Resultado esperado:

```
DIM_TIEMPO: 1461 filas generadas.
  id_tiempo  fecha  dia nombre_dia  semana  mes nombre_mes  trimestre
semestre anio es_fin_semana es_feriado
0  20220101  2022-01-01    1    Sábado     52    1      Enero         1
1  2022      1          0
1  20220102  2022-01-02    2    Domingo     52    1      Enero         1
1  2022      1          0
2  20220103  2022-01-03    3     Lunes      1    1      Enero         1
1  2022      0          0
```

PASO 3: Construcción de DIM_PRODUCTO

Generaremos 80 productos distribuidos en 4 categorías y 12 subcategorías, con atributos de marca, unidad de medida y precio de lista.

```
# --- DIM_PRODUCTO ---
categorias = {
    'Electrónica' : ['Smartphones', 'Laptops', 'Accesorios TI', 'Audio'],
    'Hogar'       : ['Muebles', 'Decoración', 'Electrodomésticos'],
    'Ropa'        : ['Hombre', 'Mujer', 'Calzado'],
    'Alimentación' : ['Abarrotes', 'Bebidas'],
}

marcas = ['Samsung', 'LG', 'Sony', 'HP', 'Lenovo', 'Asus', 'Bose',
          'IKEA', 'Mabe', 'Adidas', 'Nike', 'Gloria', 'Alicorp', 'BIC']
unidades = ['UND', 'KG', 'LT', 'PAQ', 'CJA']

filas = []
prod_id = 1
for cat, subcats in categorias.items():
    for subcat in subcats:
        n_prods = random.randint(5, 9)
        for _ in range(n_prods):
            precio = round(random.uniform(15, 2500), 2)
            filas.append({
                'id_producto' : prod_id,
                'codigo'      : f'PROD-{prod_id:04d}',
                'nombre'      : fake.catch_phrase()[:40],
                'subcategoria' : subcat,
                'categoria'   : cat,
                'marca'       : random.choice(marcas),
                'unidad_medida': random.choice(unidades),
                'precio_lista' : precio,
            })
            prod_id += 1

dim_producto = pd.DataFrame(filas)
dim_producto.to_csv('datamart_ventas/DIM_PRODUCTO.csv', index=False)
print(f'DIM_PRODUCTO: {len(dim_producto)} filas generadas.')
print(dim_producto[['id_producto', 'codigo', 'subcategoria', 'categoria', 'precio_lista']].head(
5).to_string())
```

Resultado esperado (valores aproximados):

```
DIM_PRODUCTO: 80 filas generadas.
  id_producto  codigo subcategoria categoria  precio_lista
0           1  PROD-0001   Smartphones  Electrónica      1249.90
1           2  PROD-0002   Smartphones  Electrónica       849.50
2           3  PROD-0003       Laptops  Electrónica      2189.00
```

PASO 4: Construcción de DIM_CLIENTE

Generaremos 200 clientes ficticios con datos de segmento, ciudad y tipo de cliente.

```
# — DIM_CLIENTE —
segmentos = ['Minorista', 'Mayorista', 'Corporativo', 'Gobierno']
tipos = ['Persona Natural', 'Empresa']
distritos = ['Miraflores', 'San Isidro', 'Surco', 'La Molina', 'San Borja',
             'Barranco', 'Lince', 'Magdalena', 'Jesús María', 'Pueblo Libre',
             'Trujillo', 'Arequipa', 'Cusco', 'Piura', 'Chiclayo']
provincias_map = {
    'Miraflores': 'Lima', 'San Isidro': 'Lima', 'Surco': 'Lima',
    'La Molina': 'Lima', 'San Borja': 'Lima', 'Barranco': 'Lima',
    'Lince': 'Lima', 'Magdalena': 'Lima', 'Jesús María': 'Lima',
    'Pueblo Libre': 'Lima', 'Trujillo': 'La Libertad',
    'Arequipa': 'Arequipa', 'Cusco': 'Cusco',
    'Piura': 'Piura', 'Chiclayo': 'Lambayeque'
}

filas = []
for i in range(1, 201):
    dist = random.choice(distritos)
    tipo = random.choice(tipos)
    nombre = fake.company() if tipo == 'Empresa' else fake.name()
    filas.append({
        'id_cliente' : i,
        'ruc_dni' : fake.numerify(text='#####'),
        'nombre' : nombre[:50],
        'tipo_cliente': tipo,
        'segmento' : random.choice(segmentos),
        'email' : fake.email(),
        'telefono' : fake.phone_number(),
        'distrito' : dist,
        'provincia' : provincias_map[dist],
        'departamento': provincias_map[dist] if dist not in ['Miraflores', 'San
Isidro', 'Surco', 'La Molina',
                    'San Borja', 'Barranco', 'Lince', 'Magdalena', 'Jesús
María', 'Pueblo Libre'] else 'Lima',
    })

dim_cliente = pd.DataFrame(filas)
dim_cliente.to_csv('datamart_ventas/DIM_CLIENTE.csv', index=False)
print(f'DIM_CLIENTE: {len(dim_cliente)} filas generadas.')
```

PASO 5: Construcción de DIM_TIENDA y DIM_VENDEDOR

```
# — DIM_TIENDA —————
tiendas_data = [
    (1, 'T001', 'Miraflores', 'Tienda Premium', 'Lima', 'Lima', 'Lima'),
    (2, 'T002', 'San Isidro', 'Tienda Premium', 'Lima', 'Lima', 'Lima'),
    (3, 'T003', 'Surco', 'Tienda Estándar', 'Lima', 'Lima', 'Lima'),
    (4, 'T004', 'La Molina', 'Tienda Estándar', 'Lima', 'Lima', 'Lima'),
    (5, 'T005', 'Miraflores', 'Online', 'Lima', 'Lima', 'Lima'),
    (6, 'T006', 'Trujillo', 'Tienda Estándar', 'Trujillo', 'La Libertad', 'Norte'),
    (7, 'T007', 'Arequipa', 'Tienda Premium', 'Arequipa', 'Arequipa', 'Sur'),
    (8, 'T008', 'Cusco', 'Tienda Estándar', 'Cusco', 'Cusco', 'Sur'),
    (9, 'T009', 'Piura', 'Tienda Estándar', 'Piura', 'Piura', 'Norte'),
    (10, 'T010', 'Chiclayo', 'Tienda Estándar', 'Chiclayo', 'Lambayeque', 'Norte'),
]
cols_t = ['id_tienda', 'codigo', 'nombre', 'formato', 'distrito', 'provincia', 'region']
dim_tienda = pd.DataFrame(tiendas_data, columns=cols_t)
dim_tienda.to_csv('datamart_ventas/DIM_TIENDA.csv', index=False)
print(f'DIM_TIENDA: {len(dim_tienda)} filas generadas.')

# — DIM_VENDEDOR —————
puestos = ['Asesor de Ventas', 'Vendedor Senior', 'Jefe de Tienda']
filas_v = []
for i in range(1, 31):
    filas_v.append({
        'id_vendedor' : i,
        'codigo'      : f'VEN-{i:03d}',
        'nombre'      : fake.name(),
        'puesto'      : random.choice(puestos),
        'id_tienda_asig': random.randint(1, 10),
        'activo'      : 1,
    })
dim_vendedor = pd.DataFrame(filas_v)
dim_vendedor.to_csv('datamart_ventas/DIM_VENDEDOR.csv', index=False)
print(f'DIM_VENDEDOR: {len(dim_vendedor)} filas generadas.')
```

PASO 6: Construcción de FACT_VENTAS

La tabla de hechos es el núcleo del Datamart. Cada registro representa una línea de detalle de una factura de venta. Generaremos 5,000 transacciones distribuidas aleatoriamente en el período 2022-2025, con pesos que simulan estacionalidad (mayor actividad en Noviembre-Diciembre).

```
# — FACT VENTAS —————
N_TRANSACCIONES = 5000

# Pesos de estacionalidad por mes (Dic y Nov tienen más ventas)
pesos_mes = [0.06, 0.06, 0.07, 0.07, 0.07, 0.07, 0.07, 0.08, 0.08, 0.09, 0.10, 0.12]

# Ids disponibles
ids_tiempo = dim_tiempo['id_tiempo'].tolist()
ids_producto = dim_producto['id_producto'].tolist()
ids_cliente = dim_cliente['id_cliente'].tolist()
ids_tienda = dim_tienda['id_tienda'].tolist()
ids_vendedor = dim_vendedor['id_vendedor'].tolist()

# Tabla de precios para calcular montos coherentes
precio_ref = dim_producto.set_index('id_producto')['precio_lista'].to_dict()

filas_f = []
```

```

for i in range(1, N_TRANSACCIONES + 1):
    id_prod = random.choice(ids_producto)
    precio_unit = round(precio_ref[id_prod] * random.uniform(0.85, 1.05), 2)
    cantidad = random.randint(1, 20)
    descuento_p = round(random.uniform(0, 0.25), 4) # 0% a 25%
    monto_bruto = round(precio_unit * cantidad, 2)
    descuento = round(monto_bruto * descuento_p, 2)
    monto_netto = round(monto_bruto - descuento, 2)
    costo = round(monto_netto * random.uniform(0.45, 0.70), 2)
    igv = round(monto_netto * 0.18, 2)
    margen = round(monto_netto - costo, 2)

    # Seleccionar fecha con pesos estacionales
    row_t = dim_tiempo.sample(1, weights=dim_tiempo['mes'].map(
        dict(enumerate(pesos_mes, 1))), random_state=None).iloc[0]

    filas_f.append({
        'id_venta' : i,
        'id_tiempo' : int(row_t['id_tiempo']),
        'id_producto' : id_prod,
        'id_cliente' : random.choice(ids_cliente),
        'id_tienda' : random.choice(ids_tienda),
        'id_vendedor' : random.choice(ids_vendedor),
        'nro_comprobante': f'F{random.randint(1,999):03d}-{
random.randint(1,9999):04d}',
        'nro_linea' : random.randint(1, 10),
        'cantidad' : cantidad,
        'precio_unit_sol': precio_unit,
        'monto_bruto_sol': monto_bruto,
        'descuento_sol' : descuento,
        'monto_netto_sol' : monto_netto,
        'costo_sol' : costo,
        'margen_bruto_sol': margen,
        'igv_sol' : igv,
    })

fact_ventas = pd.DataFrame(filas_f)
fact_ventas.to_csv('datamart_ventas/FACT_VENTAS.csv', index=False)
print(f'FACT_VENTAS: {len(fact_ventas)} filas generadas.')
print(f'Monto neto total: S/ {fact_ventas["monto_netto_sol"].sum():,.2f}')
print(fact_ventas[['id_venta', 'id_tiempo', 'id_producto', 'cantidad',
                    'monto_netto_sol', 'margen_bruto_sol']].head(5).to_string())

```

Resultado esperado (valores variarán según la semilla aleatoria):

```

FACT_VENTAS: 5000 filas generadas.
Monto neto total: S/ 12,487,342.50

```

	id_venta	id_tiempo	id_producto	cantidad	monto_netto_sol	margen_bruto_sol
0	1	20220115	7	3	2847.60	1138.24
1	2	20230408	45	1	153.28	78.82

PASO 7: Verificación de integridad referencial

Antes de importar a Power BI es fundamental verificar que todas las claves foráneas de FACT_VENTAS correspondan a registros existentes en las dimensiones. Ejecute el siguiente bloque de validación:

```
# --- VALIDACIÓN DE INTEGRIDAD ---
def validar_fk(fact, dim, col_fk, col_pk, nombre_dim):
    ids_fact = set(fact[col_fk].unique())
    ids_dim = set(dim[col_pk].unique())
    huerfanos = ids_fact - ids_dim
    if huerfanos:
        print(f' [ERROR] {nombre_dim}: {len(huerfanos)} FK sin correspondencia:
{list(huerfanos)[:5]}')
    else:
        cobertura = len(ids_fact) / len(ids_dim) * 100
        print(f' [OK] {nombre_dim}: todas las FK válidas. Cobertura:
{cobertura:.1f}%')

print('=== VALIDACIÓN DE INTEGRIDAD REFERENCIAL ===')
validar_fk(fact_ventas, dim_tiempo, 'id_tiempo', 'id_tiempo', 'DIM_TIEMPO' )
validar_fk(fact_ventas, dim_producto, 'id_producto', 'id_producto', 'DIM_PRODUCTO')
validar_fk(fact_ventas, dim_cliente, 'id_cliente', 'id_cliente', 'DIM_CLIENTE' )
validar_fk(fact_ventas, dim_tienda, 'id_tienda', 'id_tienda', 'DIM_TIENDA' )
validar_fk(fact_ventas, dim_vendedor, 'id_vendedor', 'id_vendedor', 'DIM_VENDEDOR')
print('=== FIN DE VALIDACIÓN ===')
```

Resultado esperado:

```
=== VALIDACIÓN DE INTEGRIDAD REFERENCIAL ===
[OK] DIM_TIEMPO : todas las FK válidas. Cobertura: 87.3%
[OK] DIM_PRODUCTO: todas las FK válidas. Cobertura: 100.0%
[OK] DIM_CLIENTE : todas las FK válidas. Cobertura: 99.5%
[OK] DIM_TIENDA : todas las FK válidas. Cobertura: 100.0%
[OK] DIM_VENDEDOR: todas las FK válidas. Cobertura: 100.0%
=== FIN DE VALIDACIÓN ===
```

PASO 8: Importar el modelo a Power BI Desktop

Con los archivos CSV generados en la carpeta datamart_ventas/, siga los pasos siguientes en Power BI Desktop:

#	Acción	Instrucción detallada
1	Abrir Power BI Desktop	Inicie Power BI Desktop. Si es la primera vez, cierre la pantalla de inicio.
2	Obtener datos	Haga clic en Inicio → Obtener datos → Texto o CSV. Navegue a la carpeta datamart_ventas/ y seleccione DIM_TIEMPO.csv. Haga clic en Cargar.
3	Repetir para cada tabla	Repita el paso anterior para: DIM_PRODUCTO.csv, DIM_CLIENTE.csv, DIM_TIENDA.csv, DIM_VENDEDOR.csv y FACT_VENTAS.csv.
4	Ir a la Vista de Modelo	En el panel izquierdo haga clic en el ícono de diagrama (Vista de Modelo). Verá las 6 tablas importadas.
5	Establecer relaciones	Arrastre el campo id_tiempo de FACT_VENTAS hacia el campo id_tiempo de DIM_TIEMPO. Repita para: id_producto, id_cliente, id_tienda, id_vendedor. Verifique que la cardinalidad sea Varios (*) a Uno (1) y la dirección de filtro cruzado sea Único.
6	Verificar el esquema estrella	El diagrama del modelo debe mostrar FACT_VENTAS en el centro rodeada por las 5 dimensiones, formando una estrella.

PASO 9: Crear medidas DAX

Las medidas DAX (Data Analysis Expressions) permiten calcular métricas dinámicas sobre el modelo. Para crear una medida: haga clic derecho sobre la tabla FACT_VENTAS en el panel de Datos → Nueva medida. Ingrese cada una de las siguientes fórmulas:

Nombre de la medida	Fórmula DAX
Total Ventas	Total Ventas = SUM(FACT_VENTAS[monto_neto_sol])
Total Costo	Total Costo = SUM(FACT_VENTAS[costo_sol])
Margen Bruto	Margen Bruto = SUM(FACT_VENTAS[margen_bruto_sol])
Margen %	Margen % = DIVIDE([Margen Bruto], [Total Ventas], 0)
Unidades Vendidas	Unidades Vendidas = SUM(FACT_VENTAS[cantidad])
Ticket Promedio	Ticket Promedio = DIVIDE([Total Ventas], COUNTROWS(FACT_VENTAS), 0)
Total Descuentos	Total Descuentos = SUM(FACT_VENTAS[descuento_sol])

PASO 10: Crear visualizaciones OLAP en Power BI

Cree las siguientes visualizaciones en una nueva página del informe llamada 'Dashboard Ventas':

#	Visualización	Campos	Operación OLAP demostrada
1	Tarjeta KPI	[Total Ventas], [Margen %], [Unidades Vendidas]	Vista general del período completo (agregación máxima)
2	Gráfico de barras apiladas	Eje X: nombre_mes Eje Y: [Total Ventas] Leyenda: categoria	Roll-Up de día → mes; desglose por categoría
3	Gráfico de líneas	Eje X: anio, trimestre, mes Eje Y: [Total Ventas]	Drill-Down de año → trimestre → mes
4	Matriz (tabla dinámica)	Filas: categoria, subcategoria Columnas: anio Valores: [Total Ventas]	Pivot y Drill-Down por jerarquía de producto
5	Mapa de árbol (Treemap)	Grupo: region Subgrupo: nombre (tienda) Valor: [Total Ventas]	Slice geográfico: distribución por región
6	Segmentador (Slicer)	DIM_TIEMPO[anio] y DIM_TIENDA[region]	Dice: filtra el cubo por año y región simultáneamente

NOTA: Para activar el Drill-Down en el gráfico de líneas, haga clic en el ícono de flecha doble (⇓) en la esquina superior del visual.

6. ACTIVIDAD PROPUESTA — 40 MINUTOS

Duración máxima	40 minutos
Modalidad	Individual o en parejas (según indicación del docente)
Entregables	(a) Código Python .py (b) Archivo Power BI .pbix (c) Capturas de pantalla de los 4 reportes solicitados

6.1 Contexto del Caso

La empresa DISTRIBUIDORA ANDINA E.I.R.L. es un mayorista de productos de construcción que opera en 6 ciudades del Perú. La gerencia de ventas necesita un Datamart Analítico de Pedidos para monitorear las ventas de sus representantes comerciales por línea de producto, cliente y período.

Usted ha sido contratado como Analista BI Jr. y debe diseñar, generar e implementar el modelo en Power BI en los 40 minutos disponibles.

6.2 Especificaciones del Modelo

El modelo debe cumplir las siguientes especificaciones:

Proceso de negocio	PEDIDOS DE CLIENTES (ventas mayoristas)
Granularidad	Una fila = una línea de pedido (1 producto × 1 cliente × 1 fecha × 1 representante)
Período	2023-01-01 al 2025-12-31
Nro. de filas en la tabla de hechos	3,000 registros generados sintéticamente

Las dimensiones requeridas son:

Dimensión	Atributos mínimos requeridos	Jerarquía que debe implementar
DIM_TIEMPO	id_tiempo, fecha, dia, mes, nombre_mes, trimestre, anio	Año → Trimestre → Mes → Día

DIM_MATERIAL	id_material, codigo, descripcion, linea, sublinea, unidad, precio_lista	Línea → Sublínea → Material
DIM_CLIENTE_MAY	id_cliente, nombre, tipo, segmento, ciudad, departamento	Departamento → Ciudad → Cliente
DIM_REPRESENTANTE	id_representante, nombre, zona, supervisor	Supervisor → Representante

Las medidas requeridas en FACT_PEDIDOS son:

- cantidad_pedida (unidades pedidas)
- precio_unitario_sol (precio de venta al mayorista)
- monto_bruto_sol (cantidad × precio_unitario_sol)
- descuento_comercial_sol (descuento por volumen, entre 0% y 20%)
- monto_netosol (monto_bruto_sol – descuento_comercial_sol)
- costo_sol (entre 55% y 75% del monto_netosol)
- margen_sol (monto_netosol – costo_sol)
- nro_pedido como dimensión degenerada (ejemplo: 'PED-001-0001')

6.3 Tareas a Resolver

Resuelva las siguientes tareas en el orden indicado:

Tarea 1: Diseño del esquema (5 min)

Dibuje (a mano alzada o en la herramienta de su preferencia) el diagrama del esquema estrella para el Datamart de Pedidos. Identifique claramente la tabla de hechos, las dimensiones, las claves primarias (PK), las claves foráneas (FK) y las medidas.

Tarea 2: Generación de datos sintéticos en Python (15 min)

Escriba el código Python completo para generar las 4 dimensiones y la tabla de hechos FACT_PEDIDOS con 3,000 registros. Utilice Faker, pandas y numpy. Exporte cada tabla a un archivo CSV dentro de una carpeta llamada datamart_pedidos/. Aplique la validación de integridad referencial para todas las FK.

Tarea 3: Implementación en Power BI (12 min)

Importe los 5 archivos CSV a Power BI Desktop. Establezca las 4 relaciones del esquema estrella (cardinalidad *→1). Cree las siguientes medidas DAX: (a) Total Pedidos = SUM(FACT_PEDIDOS[monto_netosol]) (b) Margen % = DIVIDE(SUM([margen_sol]), SUM([monto_netosol]), 0) (c) Ticket Promedio Pedido = DIVIDE([Total Pedidos], COUNTROWS(FACT_PEDIDOS), 0)

Tarea 4: Reportes solicitados (8 min)

Cree los siguientes 4 reportes en Power BI: (a) Reporte 1 — Ventas por Trimestre y Línea de Producto: Gráfico de barras agrupadas, Eje X = trimestre, Eje Y = [Total Pedidos], Leyenda = linea. Demuestre la operación Roll-Up. (b) Reporte 2 — Top 5 Representantes por Margen %: Gráfico de barras horizontal ordenado descendientemente por [Margen %], mostrando solo los top 5 representantes. (c) Reporte 3 — Tabla Dinámica: Filas = departamento, Columnas = año, Valores = [Total Pedidos]. Active el Drill-Down para navegar hasta ciudad y cliente. (d) Reporte 4 — Evolución Mensual: Gráfico de líneas con la evolución del [Total Pedidos] por mes del período 2023-2025, añadiendo un segmentador por zona del representante.

6.4 Criterios de Evaluación

#	Criterio	Puntaje	Evidencia
1	Diagrama del esquema estrella correcto y completo	2 pts	Fotografía o imagen del diagrama
2	Código Python genera datos sin errores y con integridad referencial válida	3 pts	Archivo .py o notebook
3	Modelo en Power BI con las 4 relaciones correctas y 3 medidas DAX	3 pts	Archivo .pbix
4	Los 4 reportes funcionan correctamente con Drill-Down y Slicer activos	4 pts	Capturas de pantalla
	TOTAL	12 pts	